

Звіт

Тема: «Класифікація задач. Основні поняття. Модель предметної області. Класична задача про мавпу і банан».

Вступ

Розв'язання будь-якої проблеми методами штучного інтелекту та комп'ютерного моделювання вимагає її чіткої логічної формалізації. Метою цього звіту є огляд базових понять теорії задач, принципів їх класифікації, а також дослідження процесу побудови моделі предметної області для автоматизованого планування на прикладі класичної задачі про мавпу та банан.

1. Що таке "задача": Формальне визначення

У теорії штучного інтелекту та кібернетиці задача - це не просто "проблема", це математична модель переходу від того, що ми маємо, до того, що ми хочемо отримати.

Математично (класично) будь-яка задача (від *Problem*) описується як кортеж із чотирьох елементів:

$$P = \langle S, S_0, S_G, O \rangle$$

Розшифровка термінів (Основні поняття):

S (Простір станів - State Space): Це множина *абсолютно всіх* можливих ситуацій (станів), у яких може опинитися система.

S_0 (Початковий стан - Initial State): Точка відліку. Де ми знаходимося зараз. Очевидно, що $S_0 \in S$

S_G (Цільовий стан - Goal State): Те, чого ми хочемо досягти. Це може бути один конкретний стан або множина станів. Іноді ціль задається не конкретним станом, а цільовою функцією (або предикатом), яка перевіряє, чи є поточний стан цільовим:

$$f_g(s) = \begin{cases} 1, & \text{якщо } s \in S_G \\ 0, & \text{інший випадок} \end{cases}$$

O (Оператори або Дії): Це набір правил, функцій або дій, які дозволяють переходити з одного стану в інший. Математично це функція переходу (або функція наступника), яка для заданого стану повертає множину доступних дій та

нових станів:

$$O(s) \rightarrow \{s_1, s_2, \dots, s_n\}$$

Що таке "розв'язок" задачі?

Розв'язок - це послідовність операторів (шлях у графі станів) $\{o_1, o_2, \dots, o_n\}$, яка переводить систему з S_0 в один зі станів множини S_G .

Оптимальний розв'язок - це шлях із найменшою "вартістю" (наприклад, найкоротший шлях, найменша витрата енергії чи часу). Для цього вводиться функція вартості шляху $g(x)$.

2. Класифікація задач

Задачі не бувають однаковими. Щоб комп'ютер (або людина) знав, як до неї підступитися, їх класифікують за кількома ключовими критеріями.

А. За ступенем структурованості (Класифікація Ньюелла-Саймона)

Це найголовніший поділ, який визначає, чи зможемо ми взагалі написати чіткий алгоритм.

1. Добре структуровані (Well-structured):

- **Суть:** Всі елементи $\langle S, S_0, S_G, O \rangle$ відомі, кількісно оцінені, а зв'язки між ними можна записати у вигляді точних формул.
- **Приклад:** Знайти корінь квадратного рівняння, знайти найкоротший шлях на Google Картах.
- **Як вирішують:** Класичні алгоритми, математичне моделювання, методи дослідження операцій.

2. Слабо структуровані (Ill-structured):

- **Суть:** Дані неповні, цілі конфліктні або розмиті, а правила переходу (оператори) залежать від багатьох невідомих факторів.
- **Приклад:** Медична діагностика (пацієнт каже "болить десь тут"), гра в шахи проти сильного суперника (простір станів занадто великий для повного перебору), прогнозування погоди.

- **Як вирішують:** Евристики, нейронні мережі, експертні системи, нечітка логіка (Fuzzy Logic).

3. Неструктуровані (Unstructured):

- **Суть:** Формалізувати задачу математично неможливо. Дані мають лише якісний характер.
- **Приклад:** Розробити дизайн логотипу, вирішити моральну дилему.
- **Як вирішують:** Людська інтуїція, досвід, творчість. Комп'ютери тут поки що працюють лише як допоміжний інструмент (хоча генеративний ШІ намагається залізти в цю нішу).

Б. За властивостями середовища (Де відбувається дія)

Якщо задача передбачає взаємодію з якимось середовищем (наприклад, робот рухається по складу), вона класифікується так:

- **Детерміновані vs. Стохастичні (Випадкові):**
 - *Детерміновані:* Застосування одного й того ж оператора до одного стану завжди дає однаковий результат (наприклад, хід фігурою в шахах).
 - *Стохастичні:* Є елемент випадковості (наприклад, гра в покер, де ми не знаємо, яка карта випаде наступною).
- **Статичні vs. Динамічні:**
 - *Статичні:* Середовище не змінюється, поки ми "думаємо" над рішенням (наприклад, розв'язування кросворда).
 - *Динамічні:* Середовище змінюється саме по собі (наприклад, керування безпілотним авто на жвавій вулиці).
- **Повністю спостережувані vs. Частково спостережувані:**

- *Повністю*: Ми бачимо весь простір станів (хрестики-нулики).
- *Частково*: Ми бачимо лише частину інформації (стратегія типу StarCraft, де є "туман війни").

В. За обчислювальною складністю (Класи складності)

- **Клас P**: Задачі, які можна вирішити за адекватний (поліноміальний) час.
- **Клас NP**: Задачі, вирішити які складно (перебір усіх варіантів може зайняти мільйони років), але якщо нам дадуть готове рішення, ми зможемо швидко перевірити, чи воно правильне.

3. Поняття моделі предметної області та її роль у розв'язанні задач

Розв'язання будь-якої задачі методами теорії систем або штучного інтелекту вимагає попередньої формалізації середовища, в якому ця задача розглядається. Для цього вводиться поняття моделі предметної області (МПО).

Модель предметної області - це абстрактне, формалізоване подання об'єктивної реальності або віртуального середовища, що містить виключно ті об'єкти, характеристики та закономірності, які є релевантними для розв'язання поставленої проблеми. Процес моделювання базується на абстрагуванні, тобто відкиданні несуттєвих деталей системи задля зменшення обчислювальної складності задачі.

В основі побудови МПО лежить онтологічний підхід, який структурно визначає такі компоненти:

- **Сутності (Entities)**: Базові об'єкти або агенти системи.
- **Атрибути (Attributes)**: Кількісні, якісні та просторово-часові характеристики сутностей (наприклад, координати, маса, статус).
- **Відношення (Relations)**: Логічні, просторові або причинно-наслідкові (каузальні) зв'язки між сутностями.

- **Аксиоми та правила виведення (Axioms and Rules):** Система логічних обмежень і закономірностей, що регламентують динаміку системи та допустимі переходи між її станами.

Наявність коректної та несуперечливої моделі предметної області є критичною передумовою для роботи алгоритмів пошуку рішень. Саме МПО формує базис для генерації простору станів задачі та визначення простору допустимих рішень.

4. Формалізація моделі предметної області на прикладі класичної задачі про мавпу та банан

Класична задача про мавпу та банан є парадигмальним прикладом у галузі штучного інтелекту. Її було розроблено для демонстрації принципів логічного виведення та роботи систем автоматичного планування (зокрема, алгоритму STRIPS — *Stanford Research Institute Problem Solver*). Ця задача ілюструє процеси цілеспрямованої поведінки та декомпозиції складної мети на підзадачі в межах жорстко заданої МПО.

Постановка задачі: У замкненому просторі перебуває автономний агент (мавпа). У цій же площині знаходиться допоміжний об'єкт (ящик). Цільовий об'єкт (банан) зафіксовано на висоті, що перевищує зону безпосередньої досяжності агента. Агент наділений здатністю до переміщення в просторі, маніпуляції об'єктами (штовхання ящика) та зміни власної висоти (підйом на ящик).

Етапи формалізації задачі

Для автоматизованого пошуку рішення задачу необхідно структурувати. Визначимо просторові координати як три ключові точки:

- **А** (початкова позиція агента),
- **Б** (позиція ящика) та
- **В** (позиція цільового об'єкта - банана).

1. Опис початкового стану (S_0):

Це набір стартових умов, у яких перебуває система до початку будь-яких дій.

- Агент знаходиться в точці А на рівні підлоги.
- Допоміжний об'єкт (ящик) знаходиться в точці Б.
- Цільовий об'єкт (банан) висить у точці В.
- Агент не має цільового об'єкта (мета не досягнута).

2. Опис цільового стану (S_G):

Це фінальний стан системи, до якого прагне алгоритм. Статус системи має змінитися на: «Агент володіє бананом».

3. Формалізація простору дій (Операторів):

Кожна дія агента розглядається як оператор, що переводить систему з одного стану в інший. Головне правило планування полягає в тому, що кожна дія має передумови (умови, необхідні для її виконання) та наслідки (як зміниться світ після дії).

- Оператор «Переміщення агента»:
 - *Передумова:* Агент повинен стояти на підлозі.
 - *Наслідок:* Агент змінює свої координати на потрібну точку простору.
- Оператор «Переміщення об'єкта» (Штовхати ящик):
 - *Передумова:* Агент має стояти на підлозі в одній точці з об'єктом (ящиком).
 - *Наслідок:* Агент та ящик синхронно змінюють координати на нову точку.
- Оператор «Зміна висоти» (Залізти на ящик):
 - *Передумова:* Агент та ящик знаходяться в одній точці; агент стоїть на підлозі.

- *Наслідок:* Агент змінює свій висотний рівень (перебуває на ящику).
- Оператор «Досягнення цілі» (Взяти банан):
 - *Передумова:* Агент знаходиться в координаті В (під бананом) і при цьому має відповідний рівень висоти (стоїть на ящику).
 - *Наслідок:* Зміна статусу системи на цільовий (агент володіє бананом).

Аналіз алгоритму розв'язання (Планування)

Інтелектуальна система шукає рішення за допомогою логічного виведення. Аналізуючи кінцеву мету («Взяти банан»), система бачить її головну передумову - агент має стояти на ящику в точці В. Оскільки на даний момент ящик знаходиться в точці Б, виникає логічна невідповідність.

Для усунення цієї невідповідності система формує проміжну підзадачу: перемістити ящик у точку В. Ця дія, у свою чергу, вимагає, щоб агент перебував біля ящика (в точці Б). Таким чином, система розгортає логічний ланцюжок від кінцевої мети до стартового стану, синтезуючи єдино правильний і оптимальний план дій:

1. Перейти з початкової точки А в точку Б.
2. Перештовхати ящик з точки Б у точку В (під банан).
3. Вилізти на ящик у точці В.
4. Взяти банан (ціль досягнуто).

Висновок: Цей приклад наочно доводить, що якщо правильно описати модель предметної області (виділити ключові стани, об'єкти та правила їхньої взаємодії), система отримує змогу перевести абстрактну задачу в площину чітких алгоритмічних кроків і самостійно знайти рішення.